

UNIT 3

Data Storage, Databases & Background Work

Mobile Application Development • GTU MCA Semester 3

Complete Study Notes

Simple language • Sample Java code • Diagrams • Screen outputs

■ Topics covered in this unit

1. Working with data in Android — saving state & preferences
2. Creating, saving & retrieving Shared Preferences
3. Including static files as resources
4. Working with the file system
5. Introducing Android databases — Content Values & Cursors
6. Working with SQLite databases
7. Creating Content Providers
8. Using Content Providers
9. Native Android Content Providers
10. Working in background — Services, background threads & Alarms

How to read this guide: read the **plain explanation** first, look at the **diagram / analogy**, then study the **code** and its **screen output**. Boxes in **purple** are the exam gold — memorise those.

Contents

The Big Picture (read this first!)	3
1 Working with Data — Saving State & Preferences	4
1.1 Why does an app need to “save” anything?	4
1.2 The two kinds of “state”	4
1.3 The 5 data-storage options in Android (very important table)	4
1.4 Saving temporary UI state (Activity instance state)	4
2 Creating, Saving & Retrieving Shared Preferences	5
2.1 What are Shared Preferences?	5
2.2 Getting a SharedPreferences object	6
2.3 Writing values: the Editor	6
2.4 Reading values back (always give a default!)	6
2.5 Where is the file actually stored?	7
2.6 commit() vs apply() — a favourite exam question	7
3 Including Static Files as Resources	7
3.1 What is a “static” resource?	8
3.2 The important sub-folders of res/	8
3.3 res/raw VS assets/ — know the difference	8
3.4 Example: read a text file from res/raw	8
3.5 Example: read a file from assets/	8
4 Working with the File System	9
4.1 Two storage areas: Internal vs External	9
4.2 Writing a file to internal storage	9
4.3 Reading the file back	10
4.4 File modes you should know	10
4.5 Handy folder methods	10
5 Introducing Android Databases — Content Values & Cursors	12
5.1 When do we need a database?	12
5.2 Two helper classes you will use constantly	12
5.3 ContentValues — putting data in	12
5.4 Cursor — reading data out	12
6 Working with SQLite Databases	13
6.1 The star of the show: SQLiteOpenHelper	13
6.2 Step 1 — Create the helper class	13
6.3 Step 2 — Insert (Create)	14
6.4 Step 3 — Read (Query)	14
6.5 Step 4 — Update & Delete	14
6.6 The four operations together: CRUD	15
7 Creating Content Providers	15
7.1 Why do we need Content Providers?	15
7.2 The content:// URI — the “address” of data	15
7.3 The six methods you must override	16
7.4 Skeleton of a Content Provider	16
7.5 Register it in the Manifest	17

8 Using Content Providers	17
8.1 The ContentResolver — the “client” side	17
8.2 Get the resolver and read data	17
8.3 Insert, update and delete through the resolver	18
9 Native Android Content Providers	18
9.1 Providers Android gives you for free	18
9.2 Example: read the phone’s contacts	18
9.3 Don’t forget the permission	19
10 Working in Background — Services, Threads & Alarms	20
10.1 The problem: the “UI thread” must stay free	20
10.2 Three tools for background work	20
10.3 Part 1 — Background Threads	20
10.4 Part 2 — Services	21
10.4.1 Two ways to run a Service	21
10.4.2 Service lifecycle (started service)	22
10.5 Part 3 — Alarms (AlarmManager)	22
11 Quick Revision — the whole unit on one page	24
12 Practice & Viva Questions	24
12.1 Short-answer / viva (2 marks)	24
12.2 Long-answer (5-7 marks)	25
12.3 Practical / lab tasks	25

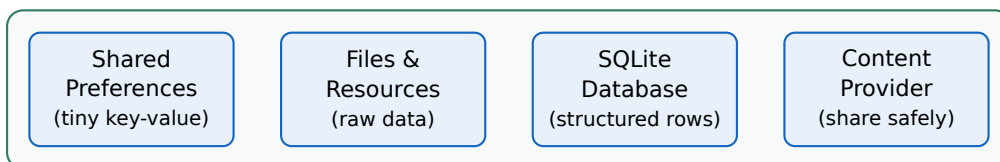
The Big Picture (read this first!)

Every real app needs to **remember things** and sometimes **do work while you are not looking**. This unit is all about those two ideas. Before we dive into topics, here is the map of the whole unit so nothing feels random.

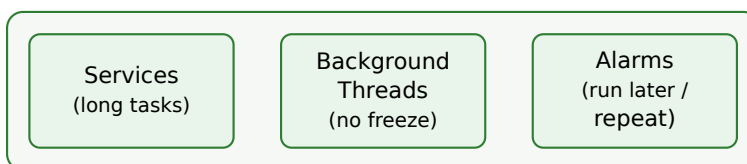
● Think of an app like a person going to office

A person needs different places to keep things: a **small diary** for tiny notes (settings), a **drawer** for files (documents), a **filing cabinet with an index** for lots of organised records (database), and a **reception desk** that lets *other* people request those records safely (content provider). And when the person leaves the desk, some **background staff** keep working (services, threads, alarms). Unit 3 teaches you each of these places, one by one.

STORING & SHARING DATA (Topics 1--9)



WORKING IN BACKGROUND (Topic 10)



✓ The one-line summary of the whole unit

Small settings → Shared Preferences. **Whole files** → File system / Resources. **Many structured records** → SQLite. **Sharing data between apps** → Content Provider. **Work without blocking the screen** → Services / Threads / Alarms.

PART A — Saving State, Preferences & Files

Topics 1 to 4: how an app remembers small settings and whole files.

Working with Data — Saving State & Preferences

Why does an app need to “save” anything?

When you rotate your phone, or press *Home* and come back, Android may **destroy and re-create** your screen (Activity). If you did not save your data, everything the user typed is **lost**. So “saving state” means: keep the data somewhere safe so the app can show it again later.

● Real-life Analogy

Imagine writing on a whiteboard. If someone wipes it clean (screen rotation / low memory), your writing is gone forever *unless* you copied it into a notebook first. Saving state = copying important things into a notebook before the board is wiped.

The two kinds of “state”

1. **Temporary UI state** — small things needed only for the current screen (text half-typed, scroll position). Saved using `onSaveInstanceState()`.
2. **Permanent data** — things that must survive even after the app is closed (login name, high score, notes). Saved using **Shared Preferences, Files, or a Database**.

The 5 data-storage options in Android (very important table)

Option	Best for	Example
Shared Preferences	Small <i>key-value</i> pairs (settings)	“remember me” = true
Internal Storage (files)	Private files only your app can read	a saved <code>notes.txt</code>
External Storage (files)	Big / shareable files (SD card)	photos, PDFs, music
SQLite Database	Many structured records (rows & columns)	a list of 500 contacts
Content Provider / Network	Sharing data with other apps or a server	Contacts app, cloud sync

✓ Key Points to Remember

Pick storage by **size and shape** of data: tiny setting → **Shared Preferences**; one whole file → **File**; many rows → **SQLite**; share with others → **Content Provider**.

Saving temporary UI state (Activity instance state)

Android calls `onSaveInstanceState(Bundle)` *just before* it destroys your Activity, and gives the same `Bundle` back in `onCreate()` OR `onRestoreInstanceState()`. A `Bundle` is simply a small box of key-value pairs.

► *MainActivity.java* — save and restore what the user typed

```

1 public class MainActivity extends AppCompatActivity {
2     EditText nameField;
3
4     @Override
5     protected void onCreate(Bundle savedInstanceState) {

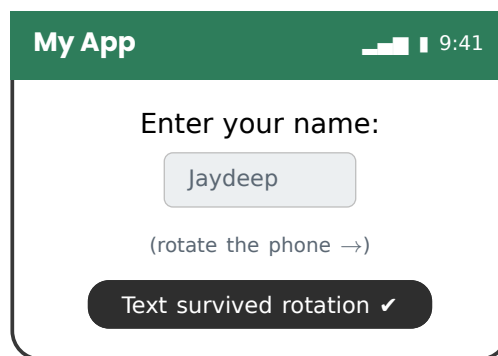
```

```

6      super.onCreate(savedInstanceState);
7      setContentView(R.layout.activity_main);
8      nameField = findViewById(R.id.nameField);
9
10     // If we saved something earlier, put it back on screen
11     if (savedInstanceState != null) {
12         String old = savedInstanceState.getString("USER_NAME");
13         nameField.setText(old);
14     }
15 }
16
17 // Called by Android BEFORE the screen is destroyed (e.g. on rotate)
18 @Override
19 protected void onSaveInstanceState(Bundle outState) {
20     super.onSaveInstanceState(outState);
21     outState.putString("USER_NAME", nameField.getText().toString());
22 }
23 }

```

Screen output: user types a name, rotates the phone, and the name is still there



⚠ Common Mistake

`onSaveInstanceState()` is **only** for small, temporary UI data. It is **not** for permanent storage — the Bundle is lost when the app is fully closed. For permanent data use Shared Preferences / Files / Database.

✓ Key Points to Remember

- `onSaveInstanceState(Bundle)` saves temporary state; the Bundle comes back in `onCreate()`.
- Data types you can put in a Bundle: String, int, boolean, float, etc.
- Rotation is the most common exam example of state loss.

Creating, Saving & Retrieving Shared Preferences

What are Shared Preferences?

Shared Preferences store small pieces of data as **key-value pairs**, exactly like a dictionary: a *key* (name) points to a *value*. Android saves them automatically in a small **XML file** inside your app's private folder. They stay even after the app is closed or the phone restarts.

● Real-life Analogy

Think of a small **diary** where each line is "Name: Value". Example: "remember_me: true", "username: jay". You open the diary, read a line, or write a new line. That diary is a Shared Preferences file.

► Shared Preferences

An interface to a small persistent XML file storing primitive key-value data (String, int, long, float, boolean). Ideal for user settings and simple flags.

Getting a SharedPreferences object

Method	When to use
getSharedPreferences("name", MODE)	You want a <i>named</i> file, sharable across many Activities.
getPreferences(MODE)	Only <i>one</i> Activity needs it (file is named after the Activity).
PreferenceManager. getDefaultSharedPreferences(ctx)	The default file used by a settings screen.

`MODE_PRIVATE` means only your own app can read the file (the normal, safe choice).

Writing values: the Editor

You cannot change preferences directly. You must open an **Editor**, put values, and then **save** with `apply()` or `commit()`.

► Saving a username and a "remember me" flag

```

1 // 1) Get the preferences file named "settings"
2 SharedPreferences prefs = getSharedPreferences("settings", MODE_PRIVATE);
3
4 // 2) Open an editor to make changes
5 SharedPreferences.Editor editor = prefs.edit();
6
7 // 3) Put key-value pairs
8 editor.putString("username", "jay");
9 editor.putBoolean("remember_me", true);
10 editor.putInt("login_count", 5);
11
12 // 4) Save! apply() = fast & background, commit() = immediate & returns true/false
13 editor.apply();

```

Reading values back (always give a default!)

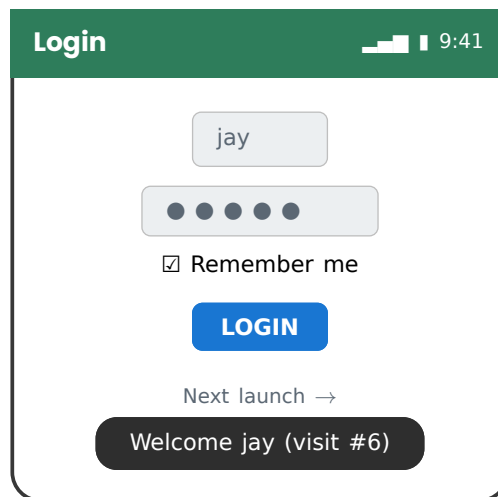
► Reading saved values with safe defaults

```

1 SharedPreferences prefs = getSharedPreferences("settings", MODE_PRIVATE);
2
3 // Second argument = default value if the key is missing
4 String name = prefs.getString("username", "Guest");
5 boolean remem = prefs.getBoolean("remember_me", false);
6 int count = prefs.getInt("login_count", 0);
7
8 welcomeText.setText("Welcome " + name + " (visit #" + count + ")");

```

Screen output: a login screen that remembers the user next time



Where is the file actually stored?

Android writes an XML file at:

/data/data/<your.package.name>/shared_prefs/settings.xml

Its content looks like this:

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <map>
3   <string name="username">jay</string>
4   <boolean name="remember_me" value="true" />
5   <int name="login_count" value="5" />
6 </map>

```

commit() vs apply() — a favourite exam question

Point	commit()	apply()
Speed / thread	Writes to disk <i>now</i> , on the main thread (can lag UI)	Writes in the <i>background</i> (does not block UI)
Return value	Returns <code>true/false</code> (success?)	Returns nothing (void)
Best when	You must know if it succeeded	Normal everyday saving (recommended)

★ Exam / Practical Tip

Use `apply()` almost always — it is faster and won't freeze the screen. Use `commit()` only when you truly need the success/fail result immediately.

✓ Key Points to Remember

- Shared Preferences = small key-value data saved as XML, survives app restarts.
- Write with `edit()` → `putXxx()` → `apply()`.
- Read with `getXxx(key, defaultValue)` — **always pass a default.**
- `MODE_PRIVATE` keeps the file readable only by your app.

Including Static Files as Resources

What is a “static” resource?

A **static file** is a file you add *while building the app* and never change at runtime — for example an image, a JSON/text data file, a sound, or a fixed layout. Android keeps all such fixed items inside two special places: the **res/** folder and the **assets/** folder.

● Real-life Analogy

Think of **res/** as a well-organised **cupboard with labelled drawers** (images drawer, layouts drawer, text drawer). Android gives every item a ticket number (the **R id**) so you can pick it instantly. **assets/** is a plain **cardboard box** where you dump files and read them by their file name.

The important sub-folders of **res/**

Folder	Holds	Accessed by
res/drawable	Images (png, jpg, vector)	R.drawable.logo
res/layout	Screen layouts (XML)	R.layout.activity_main
res/values	strings, colors, dimens, styles	R.string.app_name
res/raw	Any raw file (text, mp3, video)	R.raw.mydata
res/xml	Custom XML config files	R.xml.prefs

res/raw VS assets/ — know the difference

Both keep raw files, but they behave differently:

Point	res/raw	assets/
Access by	Resource id (R.raw.file)	File name / path (a String)
Sub-folders?	No sub-folders allowed	Yes, you can make folders
Checked at compile?	Yes (id must exist)	No (just a filename)
Read with	getResources().openRawResource()	getAssets().open("name")

Example: read a text file from **res/raw**

Put a file **welcome.txt** in **res/raw/**. Read it like this: ► *Reading res/raw/welcome.txt into a String*

```

1 InputStream is = getResources().openRawResource(R.raw.welcome);
2 BufferedReader reader = new BufferedReader(new InputStreamReader(is));
3 StringBuilder sb = new StringBuilder();
4 String line;
5 while ((line = reader.readLine()) != null) {
6     sb.append(line).append("\n");
7 }
8 reader.close();
9 textView.setText(sb.toString()); // show the file text on screen

```

Example: read a file from **assets/**

► *Reading assets/data/info.txt*

```

1 InputStream is = getAssets().open("data/info.txt"); // sub-folder allowed
2 BufferedReader reader = new BufferedReader(new InputStreamReader(is));

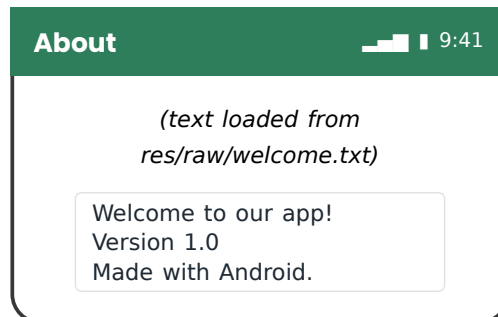
```

```

3 String firstLine = reader.readLine();
4 reader.close();
5 textView.setText(firstLine);

```

Screen output: app displays the text loaded from a bundled file



★ Exam / Practical Tip

Use `res/raw` when you want compile-time safety and a resource id. Use `assets/` when you need folders, many files, or dynamic file names (e.g. a bundled website / fonts / database).

✓ Key Points to Remember

- Static files live in `res/` (organised, id-based) or `assets/` (free-form, name-based).
- `res/raw` → `openRawResource(R.raw.x)`; `assets/` → `getAssets().open("x")`.
- `res/values/strings.xml` holds text so you can translate the app easily.

Working with the File System

Two storage areas: Internal vs External

Point	Internal storage	External storage
Who can read it	Only your app (private)	Potentially other apps / user
Space	Small	Large (SD card, big files)
Removed when	App is uninstalled	App-specific files also removed
Permission needed	None	May need permission (older Android)
Get folder with	<code>getFilesDir()</code>	<code>getExternalFilesDir(null)</code>

● Real-life Analogy

Internal storage is your **personal locker** — private, small, only you have the key. External storage is a **shared shelf** — bigger, can hold heavy items, but others may see it.

Writing a file to internal storage

The easiest way is `openFileOutput()`, which creates a file in your private folder. ► *Save some text to notes.txt (internal, private)*

```

1 String text = "Buy milk and eggs";
2 try {
3     FileOutputStream fos = openFileOutput("notes.txt", MODE_PRIVATE);
4     fos.write(text.getBytes()); // write the bytes
5     fos.close();
6     Toast.makeText(this, "Saved!", Toast.LENGTH_SHORT).show();

```

```

7 } catch (IOException e) {
8     e.printStackTrace();
9 }

```

Reading the file back

► Read *notes.txt* back into a *String*

```

1 try {
2     FileInputStream fis = openFileInput("notes.txt");
3     BufferedReader br = new BufferedReader(new InputStreamReader(fis));
4     StringBuilder sb = new StringBuilder();
5     String line;
6     while ((line = br.readLine()) != null) sb.append(line);
7     br.close();
8     noteView.setText(sb.toString());
9 } catch (IOException e) {
10     e.printStackTrace();
11 }

```

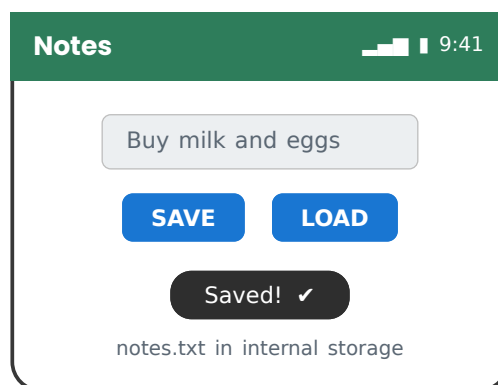
File modes you should know

Mode	Meaning
MODE_PRIVATE	Default. File is private; writing <i>replaces</i> old content.
MODE_APPEND	Add new content to the <i>end</i> of the existing file.

Handy folder methods

- `getFilesDir()` — permanent private files folder.
- `getCacheDir()` — temporary files (Android may delete when low on space).
- `getExternalFilesDir(null)` — app's private folder on external storage (no permission needed on modern Android).

Screen output: a simple notes app writes then reads a file



⚠ Common Mistake

On modern Android (10+), writing to *shared* external storage is restricted (“scoped storage”). For app-only files prefer `getExternalFilesDir()` (no runtime permission). For truly shared files use the [MediaStore](#) API or the Storage Access Framework.

✓ Key Points to Remember

- Internal = private + no permission; External = bigger + may need permission.
- Write = `openFileOutput(name, MODE)`; Read = `openFileInput(name)`.
- `MODE_PRIVATE` overwrites, `MODE_APPEND` adds to the end.
- Always wrap file I/O in try-catch and `close()` the stream.

PART B — Databases & Content Providers

Topics 5 to 9: storing many structured records, and sharing them safely between apps.

Introducing Android Databases — Content Values & Cursors

When do we need a database?

Shared Preferences and files are fine for a few settings. But what if you must store **hundreds of records** that have a fixed shape (like students, each with id, name, marks)? For that we use a **database** — data organised in **tables** of rows and columns. Android ships with **SQLite**, a tiny database built into every phone.

● Real-life Analogy

A database table is like an **Excel sheet**. Columns are the headings (Name, Age, City); each row is one record. SQLite is a mini-Excel that lives inside your app and understands the SQL language.

► SQLite

A lightweight, serverless, self-contained SQL database engine built into Android. Each app can have its own `.db` file; no separate database server is required.

Two helper classes you will use constantly

Before SQLite itself, learn the two “helper” objects that carry data *in* and *out*:

Class	Direction	Think of it as
<code>ContentValues</code>	Data going <i>INTO</i> the table	A form you fill: “column = value”
<code>Cursor</code>	Data coming <i>OUT</i> of the table	A pointer that walks over result rows

ContentValues — putting data in

`ContentValues` is a key-value bag where each key is a **column name** and each value is the data for that column. You give it to the `insert()` or `update()` methods.

► Building one row with ContentValues

```
1 ContentValues values = new ContentValues();
2 values.put("name", "Riya");    // column "name" = "Riya"
3 values.put("age", 21);        // column "age" = 21
4 values.put("city", "Rajkot");  // column "city" = "Rajkot"
5 // 'values' now represents ONE row ready to be inserted
```

Cursor — reading data out

A query returns a `Cursor`. A cursor sits *before* the first row; you move it with `moveToNext()` and read columns with `getString()`, `getInt()`, etc.

► Walking through query results with a Cursor

```
1 Cursor cursor = db.rawQuery("SELECT * FROM students", null);
2
3 while (cursor.moveToNext()) {                                // step to each row
4     String name = cursor.getString(cursor.getColumnIndex("name"));
5 }
```

```

5     int    age = cursor.getInt(cursor.getColumnIndex("age"));
6     Log.d("DB", name + " is " + age + " years old");
7 }
8 cursor.close(); // ALWAYS close the cursor when done

```

id	name	age	city
1	Riya	21	Rajkot
2	Amit	23	Surat
3	Neha	22	Baroda

Cursor
points to
current row

✓ Key Points to Remember

- `ContentValues` carries data **into** the database (column = value).
- `Cursor` carries results **out**; use `moveToNext()` then `getXxx()`.
- Always `close()` a `Cursor` to free memory.
- `getColumnIndex("col")` gives the position of a column by name.

Working with SQLite Databases

The star of the show: SQLiteOpenHelper

To use SQLite you make your own class that **extends** `SQLiteOpenHelper`. This helper does two big jobs for you automatically:

- `onCreate()` — runs **once**, the first time the DB is created. Put your “CREATE TABLE” SQL here.
- `onUpgrade()` — runs when you increase the database *version* number. Put changes (like adding a column) here.

● Real-life Analogy

`SQLiteOpenHelper` is like a **librarian**. The first day (`onCreate`) she builds the shelves. When the library is renovated (`onUpgrade`), she rearranges them. You just ask her for the books.

Step 1 — Create the helper class

► `StudentDBHelper.java`

```

1 public class StudentDBHelper extends SQLiteOpenHelper {
2
3     private static final String DB_NAME = "school.db";
4     private static final int    DB_VERSION = 1;
5
6     public StudentDBHelper(Context context) {
7         super(context, DB_NAME, null, DB_VERSION);
8     }
9
10    // Runs ONCE when the database is first created
11    @Override
12    public void onCreate(SQLiteDatabase db) {
13        db.execSQL("CREATE TABLE students ( " +
14                    "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
15                    "name TEXT, " +
16                    "age INTEGER)");
17    }
18
19    // Runs when DB_VERSION is increased

```

```
20     @Override
21     public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
22         db.execSQL("DROP TABLE IF EXISTS students");
23         onCreate(db);
24     }
25 }
```

Step 2 – Insert (Create)

► Add a new student

```
1 StudentDBHelper helper = new StudentDBHelper(this);
2 SQLiteDatabase db = helper.getWritableDatabase();
3
4 ContentValues cv = new ContentValues();
5 cv.put("name", "Riya");
6 cv.put("age", 21);
7
8 long id = db.insert("students", null, cv); // returns new row id (or -1 on error)
```

Step 3 – Read (Query)

► Read all students

```
1 SQLiteDatabase db = helper.getReadableDatabase();
2 Cursor c = db.query("students", null, null, null, null, null, "name ASC");
3
4 ArrayList<String> list = new ArrayList<>();
5 while (c.moveToNext()) {
6     list.add(c.getString(c.getColumnIndex("name"))
7             + " - " + c.getInt(c.getColumnIndex("age")));
8 }
9 c.close();
```

Step 4 – Update & Delete

► Update one row, then delete another

```
1 // UPDATE students SET age = 22 WHERE name = 'Riya'
2 ContentValues cv = new ContentValues();
3 cv.put("age", 22);
4 db.update("students", cv, "name = ?", new String[]{"Riya"});
5
6 // DELETE FROM students WHERE id = 3
7 db.delete("students", "id = ?", new String[]{"3"});
```

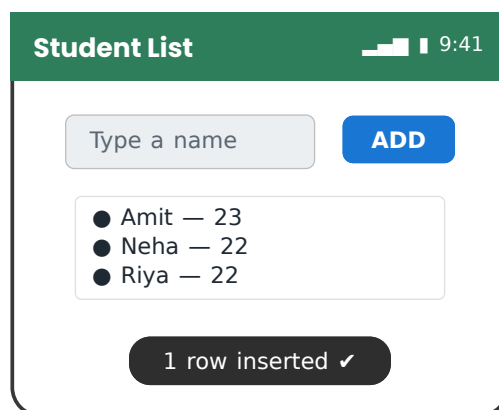
⚠ Common Mistake

Notice the ? placeholders and the `String[]` arguments. **Always** use ? instead of joining user text into SQL. Building SQL with `"..." + userInput + "..."` allows **SQL injection** attacks and crashes on names like O'Brien.

The four operations together: CRUD

Letter	SQL	SQLiteDatabase method
C — Create	INSERT	db.insert(table, null, values)
R — Read	SELECT	db.query(...) or db.rawQuery(sql, args)
U — Update	UPDATE	db.update(table, values, where, args)
D — Delete	DELETE	db.delete(table, where, args)

Screen output: a student list app backed by SQLite



✓ Key Points to Remember

- Extend `SQLiteOpenHelper`; write `CREATE TABLE` in `onCreate()`.
- `getWritableDatabase()` to change data, `getReadableDatabase()` to read.
- CRUD = insert / query / update / delete. Remember the mapping to SQL.
- Use `?` placeholders to stay safe from SQL injection, and `close()` cursors.

Creating Content Providers

Why do we need Content Providers?

Your SQLite database is **private** — other apps cannot touch it. But sometimes you *want* to share data safely (like the Contacts app shares contacts with WhatsApp). A **Content Provider** is the official “front desk” that lets other apps request your data **without** touching your database directly.

● Real-life Analogy

Your database is a **bank vault** (private). The Content Provider is the **bank teller** at the counter. Customers (other apps) never enter the vault; they ask the teller, who safely brings what they are allowed to see. This keeps your data secure.

► Content Provider

A component that manages access to a shared set of data and exposes it to other apps through a standard `ContentResolver` interface, using `content://` URIs. It is the standard way to share structured data.

The `content://` URI — the “address” of data

Every provider is reached through a special web-like address:

```
content://com.example.school.provider/students/3
```

Part	Example	Meaning
Scheme	<code>content://</code>	Always this for providers
Authority	<code>com.example.school.provider</code>	Unique name of the provider (like a domain)
Path	<code>students</code>	Which table / data set
Id (optional)	<code>3</code>	One particular row

The six methods you must override

To build a provider, extend `ContentProvider` and implement these:

Method	Job
<code>onCreate()</code>	Set up the provider (open the database helper). Runs once.
<code>query()</code>	Return rows as a <code>Cursor</code> (the READ operation).
<code>insert()</code>	Add a row; return the new item's URI.
<code>update()</code>	Change matching rows; return count changed.
<code>delete()</code>	Remove matching rows; return count deleted.
<code>getType()</code>	Return the MIME type of the data at a URI.

Skeleton of a Content Provider

► *StudentProvider.java (core structure)*

```

1 public class StudentProvider extends ContentProvider {
2
3     static final String AUTHORITY = "com.example.school.provider";
4     static final Uri CONTENT_URI =
5         Uri.parse("content://" + AUTHORITY + "/students");
6
7     private StudentDBHelper dbHelper;
8
9     @Override
10    public boolean onCreate() {
11        dbHelper = new StudentDBHelper(getContext());
12        return true; // provider ready
13    }
14
15    @Override
16    public Cursor query(Uri uri, String[] proj, String sel,
17        String[] selArgs, String sortOrder) {
18        SQLiteDatabase db = dbHelper.getReadableDatabase();
19        return db.query("students", proj, sel, selArgs, null, null, sortOrder);
20    }
21
22    @Override
23    public Uri insert(Uri uri, ContentValues values) {
24        SQLiteDatabase db = dbHelper.getWritableDatabase();
25        long id = db.insert("students", null, values);
26        return ContentUris.withAppendedId(CONTENT_URI, id);
27    }
28
29    @Override public int update(Uri u, ContentValues v, String s, String[] a){
30        return dbHelper.getWritableDatabase().update("students", v, s, a);
31    }
32    @Override public int delete(Uri u, String s, String[] a){
33        return dbHelper.getWritableDatabase().delete("students", s, a);
34    }
35    @Override public String getType(Uri uri){

```

```

36         return "vnd.android.cursor.dir/students";
37     }
38 }

```

Register it in the Manifest

A provider must be declared in `AndroidManifest.xml` so the system can find it:

```

1 <application ...>
2     <provider
3         android:name=".StudentProvider"
4         android:authorities="com.example.school.provider"
5         android:exported="true" />
6 </application>

```

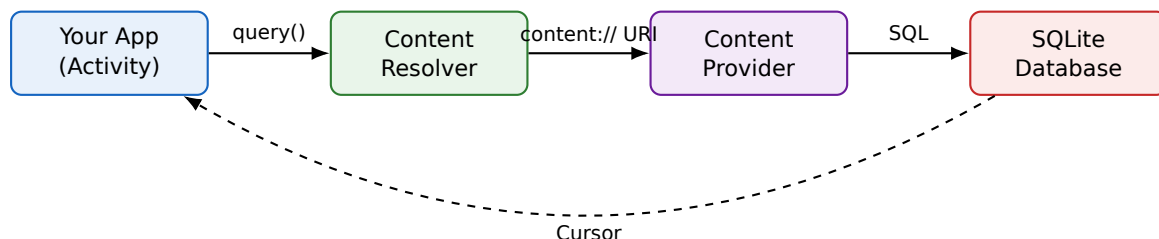
✓ Key Points to Remember

- Extend `ContentProvider` and override the **6** methods (`onCreate`, `query`, `insert`, `update`, `delete`, `getType`).
- Data address = `content://authority/path/id`.
- Declare `<provider>` with a unique `authorities` in the Manifest.
- The provider usually wraps a private SQLite database.

Using Content Providers

The ContentResolver — the “client” side

To use a provider (yours or another app’s), you never call it directly. Instead you ask the **ContentResolver**, which finds the right provider from the URI and forwards your request.



Get the resolver and read data

► Reading rows from a provider

```

1 Uri uri = Uri.parse("content://com.example.school.provider/students");
2
3 Cursor c = getContentResolver().query(
4     uri,          // which data
5     null,         // columns (null = all)
6     null, null,   // WHERE + args
7     "name ASC"); // sort
8
9 while (c.moveToNext()) {
10     String name = c.getString(c.getColumnIndex("name"));
11     // use name...
12 }
13 c.close();

```

Insert, update and delete through the resolver

► Writing through a ContentResolver

```

1 // INSERT
2 ContentValues cv = new ContentValues();
3 cv.put("name", "Sara");
4 cv.put("age", 20);
5 Uri newRow = getContentResolver().insert(uri, cv);
6
7 // UPDATE
8 cv.clear();
9 cv.put("age", 21);
10 getContentResolver().update(uri, cv, "name=?", new String[]{"Sara"});
11
12 // DELETE
13 getContentResolver().delete(uri, "name=?", new String[]{"Sara"});

```

★ Exam / Practical Tip

Same four CRUD methods, but now on `getContentResolver()` instead of `SQLiteDatabase`. The resolver is the only object your Activity needs — it hides which app actually owns the data.

✓ Key Points to Remember

- Providers are *used* through `getContentResolver()`.
- Flow: App → ContentResolver → ContentProvider → Database → Cursor back.
- The URI decides which provider handles the call.

Native Android Content Providers

Providers Android gives you for free

Android itself ships with many ready-made content providers so you can read system data (with the right permission). You use them exactly like any provider — through the ContentResolver.

Native provider	Gives access to	Permission (older Android)
Contacts (<code>ContactsContract</code>)	Phone contacts	<code>READ_CONTACTS</code>
MediaStore	Images, audio, video files	<code>READ_MEDIA_*</code> / storage
CallLog	Recent calls	<code>READ_CALL_LOG</code>
Settings	System settings values	usually none to read
CalendarContract	Calendar events	<code>READ_CALENDAR</code>

Example: read the phone's contacts

► List all contact names using the Contacts provider

```

1 Cursor c = getContentResolver().query(
2     ContactsContract.Contacts.CONTENT_URI, // built-in URI
3     null, null, null,
4     ContactsContract.Contacts.DISPLAY_NAME + " ASC");
5
6 while (c.moveToNext()) {

```

```
7     int idx = c.getColumnIndex(ContactsContract.Contacts.DISPLAY_NAME);
8     String name = c.getString(idx);
9     Log.d("CONTACT", name);
10 }
11 c.close();
```

Don't forget the permission

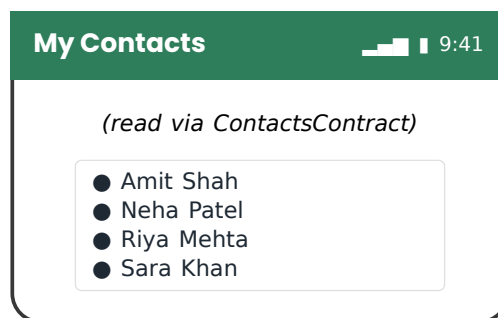
Reading contacts needs a permission in the Manifest **and** a runtime request on Android 6+:

```
1 <uses-permission android:name="android.permission.READ_CONTACTS" />
```

► Ask for the permission at runtime

```
1 if (ContextCompat.checkSelfPermission(this, Manifest.permission.READ_CONTACTS)
2     != PackageManager.PERMISSION_GRANTED) {
3     ActivityCompat.requestPermissions(this,
4         new String[]{Manifest.permission.READ_CONTACTS}, 100);
5 }
```

Screen output: app lists names pulled from the native Contacts provider



⚠ Common Mistake

Native providers hold **private user data**. You must (1) declare the permission in the Manifest, and (2) request it at *runtime* on Android 6.0+. Without the granted permission the query throws a `SecurityException`.

✓ Key Points to Remember

- Android ships built-in providers: Contacts, MediaStore, CallLog, Calendar, Settings.
- Access them with the **same** `getContentResolver().query()` using their `CONTENT_URI`.
- Sensitive providers need a declared *and* runtime-granted permission.

PART C — Working in the Background

Topic 10: doing long work without freezing the screen — Services, Threads & Alarms.

Working in Background — Services, Threads & Alarms

The problem: the “UI thread” must stay free

Everything you see (buttons, scrolling) is drawn by one special thread called the **Main** or **UI thread**. If you do slow work (download a file, big database read) *on* that thread, the screen **freezes**. If it freezes for about 5 seconds, Android shows the dreaded **ANR** (“Application Not Responding”) dialog.

● Real-life Analogy

Imagine one waiter (UI thread) serving all tables. If he stops to cook a big meal himself (slow task), every customer waits and complains (freeze / ANR). The fix: send the cooking to the **kitchen staff** (background thread / service) so the waiter keeps serving.

✓ Golden rule of Android

Never do slow work on the UI thread. Move long tasks to a background thread or a Service, and only touch UI elements back on the main thread.

Three tools for background work

Tool	Use it for	Example
Background Thread	Short slow work while the screen is open	Downloading one image
Service	Work that continues <i>even without</i> a UI	Playing music, syncing
Alarm (AlarmManager)	Running code <i>later</i> or on a schedule	Daily reminder at 8 AM

Part 1 — Background Threads

The simplest background worker is a plain Java **Thread**. But a thread **cannot** update the UI directly — you must post the result back to the main thread with `runOnUiThread()`.

► *Do slow work off the UI thread, then update UI safely*

```

1  new Thread(new Runnable() {
2      @Override
3      public void run() {
4          final String result = doSlowDownload(); // heavy work (background)
5
6          runOnUiThread(new Runnable() {           // back to UI thread
7              @Override
8              public void run() {
9                  textView.setText(result);         // safe UI update
10             }
11         });
12     }
13 }).start();

```

★ Exam / Practical Tip

Modern Android also offers easier helpers such as `AsyncTask` (older, now deprecated), `HandlerThread`, `Executors`, `WorkManager` and Kotlin `Coroutines`. For exams, remember the *rule* (no UI work off the main thread) and the `runOnUiThread()` trick.

Part 2 — Services

A **Service** is an app component that runs in the background with **no user interface**. It keeps working even after the user leaves the Activity.

► Service

An application component that performs long-running operations in the background without a UI. It keeps running even if the user switches to another app.

Two ways to run a Service

Type	Started by / lifetime	Example
Started (unbound)	<code>startService()</code> ; runs until it stops itself	Play a song, upload a file
Bound	<code>bindService()</code> ; lives while a client is bound	Activity talking to a service

► A simple started Service

```

1 public class MusicService extends Service {
2
3     @Override
4     public int onStartCommand(Intent intent, int flags, int startId) {
5         // do background work here (e.g. start playing music)
6         playMusic();
7         return START_STICKY; // restart me if the system kills me
8     }
9
10    @Override
11    public IBinder onBind(Intent intent) {
12        return null; // null = this is a started (not bound) service
13    }
14
15    @Override
16    public void onDestroy() {
17        super.onDestroy();
18        stopMusic();
19    }
20 }
```

► Start and stop the service from an Activity

```

1 startService(new Intent(this, MusicService.class)); // start
2 stopService(new Intent(this, MusicService.class)); // stop
```

Every Service must be declared in the Manifest:

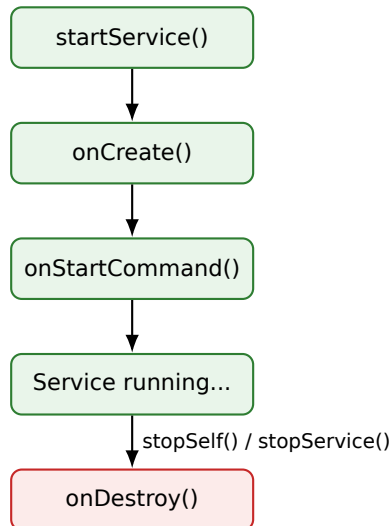
```

1 <service android:name=".MusicService" />
```

⚠ Common Mistake

A Service still runs on the **main thread** by default! For heavy work *inside* a Service, start a background thread too (or use `IntentService/JobIntentService/WorkManager`). Also, modern Android limits background services — long tasks often need a **foreground service** with a notification.

Service lifecycle (started service)



Part 3 — Alarms (AlarmManager)

An **Alarm** lets your app run code at a **future time**, even if the app is closed. Android wakes your component (usually a `BroadcastReceiver`) at the right moment. Great for reminders and periodic sync.

● Real-life Analogy

`AlarmManager` is a real **alarm clock**. You set the time and go to sleep (app closed). At that time the clock rings (fires a broadcast) and wakes up the part of your app that should act.

► Set an alarm to fire after 10 seconds

```

1  AlarmManager am = (AlarmManager) getSystemService(ALARM_SERVICE);
2
3  Intent i = new Intent(this, MyAlarmReceiver.class);
4  PendingIntent pi = PendingIntent.getBroadcast(
5      this, 0, i, PendingIntent.FLAG_IMMUTABLE);
6
7  long triggerAt = System.currentTimeMillis() + 10_000; // now + 10s
8  am.set(AlarmManager.RTC_WAKEUP, triggerAt, pi);
  
```

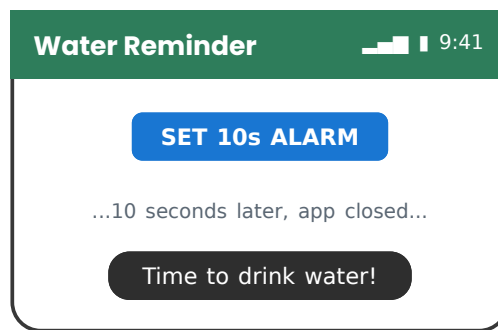
► The receiver that runs when the alarm fires

```

1  public class MyAlarmReceiver extends BroadcastReceiver {
2      @Override
3      public void onReceive(Context context, Intent intent) {
4          Toast.makeText(context, "Time to drink water!",
5              Toast.LENGTH_LONG).show();
6      }
7  }
  
```

Method	Meaning
<code>set()</code>	Fire once at a given time.
<code>setRepeating()</code>	Fire again and again at an interval.
<code>RTC_WAKEUP</code>	Use real clock time and <i>wake</i> the device if asleep.
<code>ELAPSED_REALTIME</code>	Count time since the device booted.

Screen output: a reminder alarm fires and shows a message



✓ Key Points to Remember

- UI thread must stay free $\Rightarrow$ heavy work goes to a background thread / Service.
- **Thread**: short slow work; use `runOnUiThread()` to update UI.
- **Service**: background work without UI; `onStartCommand()`; declare in Manifest.
- **AlarmManager**: run code later / repeatedly; fires a `BroadcastReceiver` even if app is closed.

Quick Revision — the whole unit on one page

✓ One-line memory hooks

Shared Preferences	small key-value settings saved as XML; <code>edit</code> → <code>put</code> → <code>apply</code> .
<code>onSaveInstanceState</code>	temporary UI state in a Bundle (survives rotation, not app close).
<code>res/</code> vs <code>assets/</code>	<code>res</code> = id-based & organised; <code>assets</code> = name-based & free-form.
Internal vs External	internal = private/no permission; external = big/may need permission.
ContentValues / Cursor SQLiteOpenHelper	values go <i>in</i> ; cursor reads <i>out</i> (<code>moveToNext</code>).
Content Provider	<code>onCreate</code> builds tables; <code>onUpgrade</code> on version change; CRUD.
Content Resolver	share data via <code>content://</code> URIs; override 6 methods.
Native providers	the client used to call any provider (<code>getContentResolver</code>). Contacts, MediaStore, CallLog, Calendar — need permission.
Background work	Thread (short) / Service (no UI) / AlarmManager (later).
Golden rule	never block the UI thread.

★ Words examiners love to hear

key-value • persistent • `MODE_PRIVATE` • `apply()` vs `commit()` • Bundle • SQLite • CRUD • ContentValues • Cursor • SQLiteOpenHelper • `content://` URI • authority • ContentResolver • MIME type • SecurityException • UI/main thread • ANR • `runOnUiThread` • `onStartCommand` • `START_STICKY` • `PendingIntent` • BroadcastReceiver • `RTC_WAKEUP`.

Practice & Viva Questions

Short-answer / viva (2 marks)

1. What is the difference between `commit()` and `apply()` in Shared Preferences?
2. Which method saves temporary UI state before an Activity is destroyed?
3. Differentiate `res/raw` and the `assets/` folder.
4. What is the difference between internal and external storage?
5. What do `ContentValues` and `Cursor` carry, and in which direction?
6. Name the two methods of `SQLiteOpenHelper` and when each runs.
7. Expand CRUD and map each letter to its `SQLiteDatabase` method.
8. What is the structure of a `content://` URI?
9. Which object is used to *call* a content provider?
10. Why must slow work not run on the UI thread? What is an ANR?
11. What is the role of `AlarmManager` and `PendingIntent`?
12. What does `START_STICKY` mean in `onStartCommand()`?

Long-answer (5--7 marks)

1. Explain the five data-storage options in Android with one example each and when to use which.
2. Write a complete example of storing and retrieving user settings using Shared Preferences.
3. Explain `SQLiteOpenHelper` and demonstrate all four CRUD operations with code.
4. What is a Content Provider? Explain the six methods and the `content://` URI with a diagram.
5. Explain how another app reads data using a `ContentResolver`. Include the request flow diagram.
6. Explain background processing in Android using Threads, Services and Alarms with examples.
7. Compare Started vs Bound services and draw the started-service lifecycle.

Practical / lab tasks

1. Build a login screen that remembers the username with a “Remember me” checkbox (Shared Preferences).
2. Make a Notes app that writes and reads a text file from internal storage.
3. Create a Student database (SQLite) supporting add, view, update and delete.
4. Wrap the Student database in a Content Provider and read it from a second Activity.
5. Read and display the phone’s contacts using the native Contacts provider (with runtime permission).
6. Set a reminder using `AlarmManager` that shows a message after 10 seconds.

You've finished Unit 3! ★

Read the **purple key-point boxes** once more before the exam, then try to *write* the small code examples from memory. That is the fastest way to score well.